



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 4 GRAFOS
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	TERCERO - B
Alumnos participantes:	Acurio López Paul Medardo
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar un grafo utilizando lista de adyacencia para representar rutas dentro del Campus Huachi de la UTA y comparar los algoritmos BFS y Dijkstra.

Específicos:

- Implementar una estructura de datos basada en listas de adyacencia en Java para modelar un grafo no dirigido y ponderado que represente ubicaciones interconectadas.
- Desarrollar el algoritmo de Búsqueda en Anchura (BFS) para identificar la ruta óptima en función del menor número de transbordos o paradas entre un punto de origen y un destino.
- Aplicar el algoritmo de Dijkstra, integrando una Cola de Prioridad, para calcular la ruta más eficiente basada en el menor costo acumulado (distancia o peso) entre nodos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

- NO modificar la estructura principal del programa.
- Completar únicamente las secciones marcadas con TODO.
- Mantener el funcionamiento correcto del programa.
- Compilar y ejecutar correctamente el archivo.
- Comparar los resultados obtenidos entre:
BFS → ruta con menos paradas.
Dijkstra → ruta con menor distancia.
Comentar el código implementado.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Apuntes de clase
- Internet
- Bibliografía virtual



TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Descripción general

La APE 4 consiste en el desarrollo de una aplicación en Java capaz de modelar un mapa simplificado (como el campus de una universidad que incluye Facultades, Biblioteca, Comedor, etc.) a través de la teoría de grafos. El programa no solo almacena los vértices (lugares) y aristas (caminos con su respectiva distancia), sino que proporciona dos mecanismos de navegación distintos: uno para encontrar la ruta con menos saltos de nodo a nodo (ideal para evitar múltiples paradas) y otro para hallar el trayecto más corto físicamente en distancia, evaluando el peso real de las conexiones.

Justificación académica

El estudio e implementación de grafos es un pilar fundamental en las Ciencias de la Computación, ya que permite la abstracción y resolución de problemas de enrutamiento y redes del mundo real. La creación manual de estos algoritmos desde cero (sin depender de librerías externas de grafos) justifica un aprendizaje profundo sobre cómo interactúan en memoria estructuras clave como las Colas, Colas de Prioridad y Mapas Hash. Además, contrastar empíricamente algoritmos de búsqueda no informada con algoritmos voraces de optimización refuerza la capacidad analítica para elegir la herramienta de software correcta según los requerimientos del negocio.

Explicación de la Estructura Utilizada

El grafo fue diseñado utilizando una Lista de Adyacencia soportada por mapas clave-valor.

- **Map<String, Nodo> nodos:** Actúa como un diccionario que guarda la información (ID y Nombre) de cada vértice para un acceso instantáneo de orden $O(1)$.
- **Map<String, List<Arista>> adyacencia:** Asocia el identificador de cada nodo con una lista dinámica de aristas (las cuales contienen el destino y el peso de la conexión).

Se eligió esta estructura por encima de una "Matriz de Adyacencia" porque, en la mayoría de sistemas de rutas, los grafos son dispersos (no todos los nodos están conectados con todos). Esto optimiza drásticamente el consumo de memoria y agiliza las iteraciones sobre los "vecinos" de un nodo.

Definición de Conceptos Clave

- **Grafo No Dirigido Ponderado:** Una red de nodos donde las conexiones son bidireccionales (se puede ir de A a B y viceversa con el mismo costo) y cada conexión posee un valor numérico (peso/distancia).



- **BFS (Breadth-First Search):** Algoritmo que explora un grafo "por niveles" concéntricos. Garantiza encontrar el camino más corto en grafos no ponderados, o en este caso, la ruta con el menor número de nodos intermedios.
- **Algoritmo de Dijkstra:** Algoritmo diseñado para encontrar el camino de costo mínimo desde un vértice origen al resto de los vértices en un grafo ponderado (con pesos positivos).
- **Priority Queue (Cola de Prioridad):** Estructura de datos abstracta utilizada en Dijkstra que, en lugar de atender elementos por orden de llegada (FIFO), siempre desencola el elemento que posee la mayor prioridad (en este caso, la menor distancia acumulada).

Análisis de Complejidad Temporal

- Algoritmo BFS: Su complejidad temporal es de $O(V + E)$, donde V es el número de vértices (nodos) y E el número de aristas. Esto se debe a que, en el peor de los casos, el algoritmo encola y desencola cada vértice una vez e inspecciona cada arista adyacente una vez.
- Algoritmo de Dijkstra: Gracias a la implementación de la PriorityQueue (basada en un montículo binario en Java), extraer el nodo con la distancia mínima cuesta $O(\log V)$. Dado que este proceso se repite para los V vértices y se actualizan E aristas, la complejidad temporal se establece en $O((V + E) \log V)$.

2.7 Resultados obtenidos

Evidencia de Ejecución

El desarrollo colaborativo y los entregables se gestionan a través de control de versiones.

- **Código en JAVA**

```
1  import java.util.*;
2
3  public class Main {
4
5      // =====
6      // Nodo
7      // =====
8      static class Nodo { 3 usages
9          String id; 2 usages
10         String nombre; 2 usages
11
12         @Contract(pure = true)
13         public Nodo(String id, String nombre) { 1 usage
14             this.id = id;
15             this.nombre = nombre;
16         }
17     }
18
19     // =====
20     // Arista
21     // =====
22     static class Arista { 5 usages
23         String destino; 9 usages
24         int peso; 2 usages
25
26         @Contract(pure = true)
27         public Arista(String destino, int peso) { 2 usages
28             this.destino = destino;
29             this.peso = peso;
30         }
31     }
32 }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
30
31 // =====
32 // Grafo
33 // =====
34 static class Grafo { 2 usages
35
36     Map<String, Nodo> nodos = new HashMap<>(); 3 usages
37     Map<String, List<Arista>> adyacencia = new HashMap<>(); 5 usages
38
39     // =====
40     // TODO 1
41     // Agregar nodo al grafo
42     // =====
43     public void agregarNodo(String id, String nombre) { 6 usages
44
45         // COMPLETE AQUÍ
46         nodos.put(id, new Nodo(id, nombre));
47         adyacencia.putIfAbsent(id, new ArrayList<>());
48     }
49
50     // =====
51     // TODO 2
52     // Agregar arista no dirigida
53     // =====
54     public void agregarArista(String origen, String destino, int peso) { 6 usages
55
56         // COMPLETE AQUÍ
57         adyacencia.get(origen).add(new Arista(destino, peso));
58         adyacencia.get(destino).add(new Arista(origen, peso));
59     }
60
61 }
62
63 // =====
64 // TODO 3 – BFS
65 // Ruta con menos paradas
66 // =====
67 public List<String> bfs(String inicio, String fin) { 1 usage
68
69     // Cola para recorrer niveles
70     Queue<List<String>> cola = new LinkedList<>();
71
72     // Nodos visitados
73     Set<String> visitados = new HashSet<>();
74
75     // Camino inicial
76     List<String> caminoInicial = new ArrayList<>();
77
78     // TODO:
79     // Agregar nodo inicio al camino inicial
80     caminoInicial.add(inicio);
81
82     // TODO:
83     // Agregar caminoInicial a la cola
84     cola.add(caminoInicial);
85
86     // TODO:
87     // Marcar inicio como visitado
88     visitados.add(inicio);
89
90     while (!cola.isEmpty()) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
91
92 // TODO:
93 // Obtener el primer camino de la cola
94 List<String> camino = cola.poll();
95
96 // Nodo actual
97 String actual =
98     camino.get(camino.size() - 1);
99
100 // Si llegamos al destino
101 if (actual.equals(fin)) {
102     return camino;
103 }
104
105 // Recorrer vecinos
106 for (Arista arista : adyacencia.get(actual)) {
107
108     // TODO:
109     // Verificar si el vecino NO fue visitado
110     if (!visitados.contains(arista.destino)) {
111
112         // TODO:
113         // Marcar vecino como visitado
114         visitados.add(arista.destino);
115
116         // Crear nuevo camino
117         List<String> nuevoCamino =
118             new ArrayList<>(camino);
119
120         // TODO:
121         // Agregar vecino al nuevo camino
122         nuevoCamino.add(arista.destino);
123
124         // TODO:
125         // Agregar nuevoCamino a la cola
126         cola.add(nuevoCamino);
127     }
128 }
129
130
131 return null;
132 }
133
134 // =====
135 // TODO 4 - Dijkstra
136 // Ruta con menor distancia
137 // =====
138 public List<String> dijkstra(String inicio, String fin) { 1usage
139
140     Map<String, Integer> distancias =
141         new HashMap<>();
142
143     Map<String, String> anteriores =
144         new HashMap<>();
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
145
146     PriorityQueue<String> cola =
147         new PriorityQueue<>(
148             Comparator.comparingInt(
149                 distancias::get
150             )
151         );
152
153     // Inicializar distancias
154     for (String nodo : nodos.keySet()) {
155
156         // TODO:
157         // Inicializar distancia infinita
158         distancias.put(nodo, Integer.MAX_VALUE);
159     }
160
161
162     // TODO:
163     // Distancia del inicio = 0
164     distancias.put(inicio, 0);
165
166     // TODO:
167     // Agregar inicio a la cola
168     cola.add(inicio);
169
170     while (!cola.isEmpty()) {
171
172         // TODO:
173         // Obtener nodo con menor distancia
174         String actual = cola.poll();
175
176         // Si la distancia al nodo actual es infinito, se rompe (nodos inalcanzables)
177         if (distancias.get(actual) == Integer.MAX_VALUE) break;
178
179         for (Arista arista : adyacencia.get(actual)) {
180
181             // TODO:
182             // Calcular nueva distancia
183             int nuevaDistancia = distancias.get(actual) + arista.peso;
184
185             // TODO:
186             // Verificar si nuevaDistancia es menor
187             if (nuevaDistancia < distancias.get(arista.destino)) {
188
189                 // TODO:
190                 // Actualizar distancia
191                 distancias.put(arista.destino, nuevaDistancia);
192
193                 // TODO:
194                 // Guardar nodo anterior
195                 anteriores.put(arista.destino, actual);
196
197                 // TODO:
198                 // Agregar vecino a la cola
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
199         // Removemos y agregamos para que la PriorityQueue se reordene correctamente
200         cola.remove(arista.destino);
201         cola.add(arista.destino);
202     }
203 }
204 }
205
206 // Reconstruir camino
207 List<String> camino = new ArrayList<>();
208
209 String actual = fin;
210
211 // Si el destino es inalcanzable
212 if (!anteriores.containsKey(fin) && !fin.equals(inicio)) {
213     return null;
214 }
215
216 while (actual != null) {
217
218     camino.add(index: 0, actual);
219
220     actual = anteriores.get(actual);
221 }
222
223 return camino;
224 }
225
```

```
226 // =====
227 // Mostrar resultado
228 // =====
229 public void mostrarRuta(List<String> ruta) { 2 usages
230
231     if (ruta == null) {
232         System.out.println("No existe ruta");
233         return;
234     }
235
236     for (int i = 0; i < ruta.size(); i++) {
237
238         String idNodo = ruta.get(i);
239
240         Nodo nodo = nodos.get(idNodo);
241
242         System.out.print(
243             nodo.nombre + " (" + nodo.id + ")"
244         );
245
246         if (i < ruta.size() - 1) {
247             System.out.print(" -> ");
248         }
249     }
250
251     System.out.println();
252 }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
253     }
254
255     // =====
256     // MAIN
257     // =====
258     public static void main(String[] args) {
259
260         Grafo grafo = new Grafo();
261
262         // NODOS
263         grafo.agregarNodo( id: "uta", nombre: "Universidad");
264         grafo.agregarNodo( id: "fisei", nombre: "FISEI");
265         grafo.agregarNodo( id: "idiomas", nombre: "Idiomas");
266         grafo.agregarNodo( id: "biblioteca", nombre: "Biblioteca");
267         grafo.agregarNodo( id: "estadio", nombre: "Estadio");
268         grafo.agregarNodo( id: "comedor", nombre: "Comedor");
269
270         // ARISTAS
271         grafo.agregarArista( origen: "uta", destino: "fisei", peso: 50);
272         grafo.agregarArista( origen: "fisei", destino: "idiomas", peso: 40);
273         grafo.agregarArista( origen: "idiomas", destino: "biblioteca", peso: 30);
274         grafo.agregarArista( origen: "biblioteca", destino: "estadio", peso: 70);
275
276         // Ruta con menos paradas
277         // pero más distancia
278         grafo.agregarArista( origen: "uta", destino: "comedor", peso: 20);
279         grafo.agregarArista( origen: "comedor", destino: "estadio", peso: 200);
280
281         // =====
282         // PRUEBAS
283         // =====
284
285         System.out.println("==== BFS ====");
286
287         List<String> rutaBFS =
288             grafo.bfs( inicio: "uta", fin: "estadio");
289
290         grafo.mostrarRuta(rutaBFS);
291
292         System.out.println("\n==== DIJKSTRA ====");
293
294         List<String> rutaDijkstra =
295             grafo.dijkstra( inicio: "uta", fin: "estadio");
296
297         grafo.mostrarRuta(rutaDijkstra);
298     }
299 }
```

- **Ejecución**

```
Run Main x
C:\Users\User\jdk\openjdk-25\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2\lib\idea_rt.jar=52998" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=
==== BFS ====
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)

==== DIJKSTRA ====
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)

Process finished with exit code 0
```

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía



- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se comprobó que el criterio de optimización altera completamente el resultado: BFS seleccionó una ruta con un solo intermediario (Comedor) ignorando su alta distancia, mientras que Dijkstra encontró una ruta con más paradas, pero con una distancia total significativamente menor.
- La implementación mediante diccionarios (HashMap) resultó ser altamente flexible para manejar identificadores de nodos en formato String, haciendo el código más legible e intuitivo.

2.10 Recomendaciones

- Para proyectos futuros que escalen a miles o millones de nodos (como el mapa de una ciudad), se recomienda reemplazar la PriorityQueue estándar por un Fibonacci Heap, lo que reduciría la complejidad teórica de Dijkstra a $O(E + V \log V)$.
- Se sugiere agregar validaciones para capturar excepciones en caso de que un usuario ingrese un nodo de origen o destino que no haya sido registrado previamente en el grafo.

2.11 Referencias bibliográficas

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, Introduction to Algorithms, 3.^a ed. Cambridge, MA, EE. UU.: MIT Press, 2009.
- [2] R. Sedgewick y K. Wayne, Algorithms, 4.^a ed. Upper Saddle River, NJ, EE. UU.: Addison-Wesley Professional, 2011.
- [3] M. A. Weiss, Data Structures and Algorithm Analysis in Java, 3.^a ed. Boston, MA, EE. UU.: Pearson, 2011.

2.12 Anexos

Enlace de GitHub: <https://github.com/Paul-Acurio/Ape4-Grafos.git>